Recover the Design

2026-11-03

Abstract

This project reconstructs an unknown fence layout from connector degrees. Fences may span empty positions, while horizontal and vertical fences may cross only at a connector. We model every possible straight fence between consecutive non-zero connectors and solve a degree-constrained selection problem with pairwise crossing conflicts. The exact solver combines static checks, event-driven degree propagation, immediate conflict rejection, a rollback trail, and minimum-remaining-values (MRV) branching. Correctness is checked against exhaustive small grids and an independent validator. Across a deterministic 24-case benchmark corpus, all cases completed within the 30-second safety timeout. The maximum observed median runtime was 331.9 ms. Conflict propagation reduced search from 39,888 to 162 nodes on the largest planted case, while MRV was beneficial on medium conflict-heavy inputs but not uniformly faster.

Keywords: recovering a layout; degree constraints; event-driven propagation; MRV; conflict graph; exact search

Contents

1 Introduction **3**
1.1 Problem and objectives . . . 3
1.2 Why the geometry requires a conflict model . . . 3

2 Data Structure and Algorithm Specification **3**
2.1 Candidate graph . . . 3
2.2 Exact constraints . . . 3
2.3 Static checks . . . 3
2.4 Event-driven propagation . . . 4
2.5 MRV branching . . . 4
2.6 Search and rollback . . . 4

3 Testing Results **5**
3.1 Environment and method . . . 5
3.2 Correctness cases . . . 5
3.3 Benchmark corpus . . . 6

4 Analysis and Comments **6**
4.1 Complexity . . . 6
4.2 Version comparison . . . 6
4.3 Comments and limitations . . . 7
4.4 Further improvements . . . 7

A Source Code and Reproduction **7**

1 Introduction

1.1 Problem and objectives

The input is an $n \times m$ grid. A positive value $k \in \{1, 2, 3, 4\}$ denotes a connector that must be incident to exactly k fence segments. A zero means that no connector exists. Each fence is a straight horizontal or vertical segment. A fence may pass through empty positions, but horizontal and vertical fences may cross only when the crossing position contains a connector.

The program must output the binary `up/down/left/right` states of every connector in row-major order, or print `No Solution`. Solutions are not unique, so any valid layout is accepted. The objective is to find a valid layout or prove that none exists.

1.2 Why the geometry requires a conflict model

A common but incorrect simplification is to connect only adjacent non-zero cells. Sample 3 disproves it: the connector in row 1, column 3 connects to the connector in row 1, column 5 across an empty column. Therefore every possible fence is a maximal straight segment between two consecutive non-zero connectors in one row or one column.

Two such segments can cross inside an empty cell. If both were selected, they would form a crossing without a connector, which the statement forbids.

2 Data Structure and Algorithm Specification

2.1 Candidate graph

Let V be the set of non-zero connectors and E the set of candidate fence segments. In each row, sorted non-zero columns define horizontal segments between consecutive connectors. In each column, sorted non-zero rows define vertical segments analogously. For a connector v, let $E_v \subseteq E$ be the set of incident candidates; $|E_v| \leq 4$.

For each possible horizontal/vertical pair (e, f), a conflict exists when the two segments intersect strictly inside an empty cell. The conflict relation is stored as a symmetric adjacency list. Because candidate intervals in one row or column do not overlap, each empty cell contains at most one horizontal and one vertical candidate, hence at most one conflict pair.

2.2 Exact constraints

Let $x_e \in \{0, 1\}$ indicate whether candidate e is selected. A valid layout satisfies

$$\sum_{e \in E_v} x_e = d_v \qquad \text{for every } v \in V, \tag{1}$$

$$x_e + x_f \leq 1 \qquad \text{for every conflict pair } (e, f). \tag{2}$$

Every selected segment contributes one degree to each endpoint. Thus the total number of selected segments is exactly $\frac{1}{2} \sum_v d_v$.

2.3 Static checks

The model is rejected before search when:

- the total requested degree is odd;
- a connector has fewer incident candidates than its required degree;
- a candidate-graph component has odd degree demand or too few edges.

These checks are necessary conditions only. Passing them does not imply that a solution exists.

2.4 Event-driven propagation

The solver stores an edge state in $\{\text{UNKNOWN}, \text{SELECTED}, \text{REJECTED}\}$. For each connector it maintains the number of selected incident edges and the number of unknown incident edges. When an edge changes state, both endpoint connectors enter an event queue.

For a queued connector v, let s_v be its selected degree, u_v its unknown degree, and d_v its required degree. The branch fails immediately if $s_v > d_v$ or $s_v + u_v < d_v$. Otherwise:

- if $s_v = d_v$, every unknown incident edge is rejected;
- if $s_v + u_v = d_v$, every unknown incident edge is selected.

Selecting an edge immediately rejects all crossing candidates. Those changes enqueue their endpoints, so the propagation continues until a fixed point is reached or a contradiction is found.

Listing 1: Incremental propagation at one connector

```
while queue is not empty:
    v = pop_vertex(queue)
    if selected[v] > required[v]: reject_branch()
    if selected[v] + unknown[v] < required[v]: reject_branch()

    if selected[v] == required[v]:
        reject_all_unknown_edges_incident_to(v)
    else if selected[v] + unknown[v] == required[v]:
        select_all_unknown_edges_incident_to(v)
```

2.5 MRV branching

After propagation reaches a fixed point, every unsatisfied connector has

$$need_v = d_v - s_v, \qquad B_v = \binom{u_v}{need_v}. \tag{3}$$

B_v is the number of immediate local assignments for connector v. MRV chooses the connector with the smallest B_v; since $u_v \le 4$, $B_v \le 6$. Ties prefer more incident crossing conflicts and then the smaller internal vertex id. MRV changes only the order of complete value enumeration and therefore preserves completeness.

2.6 Search and rollback

At each search level, propagation is completed first. If every connector has exactly its required degree, the first solution is returned. Otherwise MRV selects a connector and enumerates every combination of exactly $need_v$ unknown incident edges. The chosen edges are selected and the other incident unknown edges are rejected. Every state change records its edge id and previous state in a trail.

If a branch fails, the trail is rolled back to the branch checkpoint, restoring edge states and both degree counters. The next combination is then tried. The search is complete because every legal completion must choose one of the enumerated local subsets.

3 Testing Results

3.1 Environment and method

Table 1: Experimental environment

Item	Configuration
Platform	Windows, Intel CPU
Compiler	MSVC 19.51
Build system	CMake, Visual Studio 18 2026 generator
Configurations	Debug and Release
Correctness	CTest plus independent output validator
Benchmark repetitions	one warm-up, then five measured runs

3.2 Correctness cases

Table 2: Correctness test summary

Test group	Cases	Status	Purpose
Official samples	3	pass	Feasible, impossible, multi-solution
Exhaustive 2×2	625	pass	Every degree matrix
Exhaustive 2×3	15,625	pass	Complete small-input differential test
Random 2×2 to 3×4	1,000	pass	Brute-force differential test
Random 3×3 and 3×4	3,000	pass	Larger differential test
Planted feasible layouts	200	pass	No false `No Solution`
Required edge cases	9	pass	Straight, T, cross, boundary, disconnected
Forced propagation	1	pass	Queue and forced selection executed
Conflict propagation	1	pass	Crossing rejection executed

Every constructed answer is passed back through the independent validator. Debug and Release CTest both pass. Invalid degree, malformed output, one-sided fences, and empty-cell crossings are rejected by dedicated tests.

3.3 Benchmark corpus

Table 3: Current solver results by category

Category	Cases	Max median (ms)	Max nodes
Conflict-heavy 50x50	1	6.826	205
Dense all-positive 50x50	1	332.151	2,402
Fragmented 10/25/50	3	0.216	8
Infeasible boundary	1	0.001	0
Infeasible odd degree	3	0.360	0
Planted feasible 10/25/50	3	12.322	1,404
Random dense 10/25/50	3	0.498	0
Random sparse p=0.10	3	0.106	0
Random sparse p=0.25	3	0.246	0
Random sparse p=0.50	3	0.395	0

All 24 cases completed. The maximum measured run was 349.880 ms, well below the 30-second safety timeout. All search statistics were identical across five repetitions.

4 Analysis and Comments

4.1 Complexity

Let P be the number of connectors and M the number of candidate fence segments. Here $P \leq 2500$ and $M < 5000$. Building the connectors and candidate edges takes $O(nm + P)$. Scanning the empty cells for conflicts takes $O(nm + M)$, and the conflict graph contains at most $O(nm)$ pairs.

One propagation event processes at most four incident edges, but rejected conflict edges may enqueue their own endpoints. If W changes occur on one search path, propagation costs $O(W)$ in that branch. Selecting the MRV variable scans P connectors and inspects four incident edges each, so one selection costs $O(P)$.

If N is the number of visited search nodes and L is the cost of one node, the search takes $O(NL)$. Branching remains exponential in the worst case. It is therefore necessary to report measured node counts rather than only the theoretical upper bound.

4.2 Version comparison

Table 4: Search-node comparison on representative cases

Case	Degree only	Degree+conflict	MRV	MRV change
Planted 10x10	21	14	7	-50.0%
Planted 25x25	176,118	6,636	137	-97.9%
Planted 50x50	39,888	162	1,404	+766.7%
Conflict-heavy 50x50	271	265	205	-22.6%
Dense all-positive 50x50	2,402	2,402	2,402	0.0%

The no-propagation baseline completed 20 of 24 cases within a five-second comparison limit; four large cases timed out. Degree-only propagation completed all cases, while adding conflict propa-

gation sharply reduced nodes on planted inputs. MRV improved the medium planted and conflict-heavy cases but selected an unlucky branch on the largest planted case.

4.3 Comments and limitations

MRV is a heuristic, not a guarantee of a smaller global subtree. The dense all-positive graph has the same node count under fixed order and MRV, but MRV incurs a full connector scan at every node. This overhead is visible in the roughly 332 ms median against about 3 ms for fixed order.

The official problem statement gives no time limit. Results therefore describe this machine, compiler, seed, and input corpus. A separate 50x50 planted probe with density 0.50–0.55 exceeded a 15–30 second limit and is excluded from the fixed acceptance corpus. This identifies dense random degree-constrained instances as a limitation rather than claiming universal efficiency.

4.4 Further improvements

Future work can maintain the unresolved-connector set to avoid scanning all connectors for MRV, add stronger forward checking after each local combination, and use component decomposition when conflict edges do not couple candidate components. A certified maximum bipartite subgraph solver could also provide cheaper infeasibility bounds on bipartite components.

A Source Code and Reproduction

Table 5: Source organization

File	Role
`io.cpp/hpp`	Input parsing and row-major output
`model.cpp/hpp`	Candidate graph and conflict construction
`solver.cpp/hpp`	Propagation, MRV, rollback, complete search
`validator.cpp/hpp`	Independent answer validation
`tests/test_runner.cpp`	Samples, exhaustive and randomized tests
`tests/benchmark_runner.cpp`	Reproducible CSV benchmark runner

Build and run:

Listing 2: Build and execution commands

```
cmake -S . -B build -G "Visual␣Studio␣18␣2026" -A x64
cmake --build build --config Release
./build/Release/recover.exe < input.txt
ctest --test-dir build -C Release --output-on-failure
```

References

[1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, fourth edition, 2020.

[2] D. E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms*. Addison-Wesley, 2011.

Author List

Author identities and contribution details are withheld in this anonymous peer review version.

Declaration

We hereby declare that all work done in this project is of our independent effort as a group.